

接入文档-iOS

喜马拉雅生态SDK iOS集成指南

概述

喜马拉雅生态SDK (XMEcoSDK) 为第三方应用提供音频内容服务，主要能力包括：音频播放控制、WKWebView 内容接入、用户认证、悬浮播放器、锁屏播控、数据获取等。

系统要求

- iOS 13.0+ / Xcode 16+ / Swift 5.9+

1. 集成步骤

1.1 添加依赖

1. 将 `XMEcoSDK.xcframework` 拖入工程，勾选 **Embed & Sign**
2. 将 `Sources/XMEcoSDK/` (Swift 悬浮窗文件: `FloatingPlayerView.swift` / `FloatingPlayerIcons.swift` / `FloatingPlayerManager.swift`) 加入 target
渠道签名密钥 (appKey/appSecret/sn) 已在 xcframework 编译期通过 cinterop 静态链接 `SignatureLib`，接入方无需单独引入 `SignatureLib`。
xcframework 内含真机 slice (`ios-arm64`) 与模拟器 slice (`ios-arm64_x86_64-simulator`)，覆盖 Apple Silicon 与 Intel Mac 模拟器)，Xcode 按 destination 自动选择，真机/模拟器调试均无需额外配置。

1.2 权限配置

Info.plist 放开 ATS (H5 与音频流需要)：

▽

复制代码

```
<key>NSAppTransportSecurity</key>
<dict>
  <key>NSAllowsArbitraryLoads</key><true/>
</dict>
```

Signing & Capabilities → Background Modes → 勾选 **Audio, AirPlay, and Picture in Picture**。

1.3 初始化

渠道密钥已内置在签名库中，初始化无需传参：

```
func application(_ application: UIApplication,
                 didFinishLaunchingWithOptions launchOptions: [...]) ->
Bool {
    XMEcoSDK.shared.initialize()

    FloatingPlayerManager.shared.install() // 注册悬浮窗（App 启动调一次）
    FloatingPlayerManager.shared.h5PlayerNavigator = { url, presenter in
        let vc = WebViewController(); vc.url = url
        presenter?.navigationController?.pushViewController(vc,
animated: true)
    }

    window = UIWindow(frame: UIScreen.main.bounds)
    window?.rootViewController =
    UINavigationController(rootViewController: MainViewController())
    window?.makeKeyAndVisible()
    return true
}
```

日志开关（默认关闭，需要时显式开启）：iOS 无法在运行时可靠识别宿主 App 是测试包还是正式包，故默认 WARN（仅告警/错误，URL 中的 `access_token` / `mobile` 等敏感参数自动脱敏为

`***`）。联调需要全量日志时传 `debug: true`，推荐跟随工程配置：

```
#if DEBUG
    XMEcoSDK.shared.initialize(debug: true) // 测试构建：DEBUG 全量日志
#else
    XMEcoSDK.shared.initialize() // 正式构建：WARN 上线安全
#endif
```

2. WebView 集成

复制代码

```
class WebViewController: UIViewController, BridgeEventListener {
    func viewDidLoad() {
        super.viewDidLoad()
        XMEcoSDK.shared.setEventListener(listener: self) // 监听 H5 事件

        let webView = WKWebView(frame: view.bounds)
        view.addSubview(webView)
        // attachWebView 内聚了 navigationDelegate, 自动拦截
        weixin://alipays:// 等支付 scheme 拉起外部 App
        XMEcoSDK.shared.attachWebView(webView: webView,
        userContentController: userContentController)
        webView.load(URLRequest(url: URL(string:
        SdkUrls.shared.H5_BASE_URL)!))
    }

    func onEvent(event: BridgeEvent) {
        if event.module == "UtilBridge" && event.method == "closePage" {
            navigationController?.popViewController(animated: true)
        }
    }
}
```

2.1 容器标题栏适配 (URL 判断)

SDK 内的 H5 页面（播放器、专辑详情、权益等）自带标题栏；外部页面（支付中间页等）不带标题栏且无 bridge 适配。容器在页面加载时按 URL 判断：URL 含 `H5_BASE_URL` 的为 SDK 页面 → 隐藏原生标题栏（H5 自带）；其余为外部页面 → 显示原生标题栏。纯容器行为，H5 侧零适配。

复制代码

```
// WKNavigationDelegate.decidePolicyFor 里按 URL 粗判
func webView(_ webView: WKWebView, decidePolicyFor navigationAction:
WKNavigationAction,
        decisionHandler: @escaping (WKNavigationActionPolicy) ->
Void) {
    let url = navigationAction.request.url?.absoluteString ?? ""
```

```
titleBarView.isHidden = url.hasPrefix(SdkUrls.shared.H5_BASE_URL)
decisionHandler(.allow)
```

返回按钮建议：优先 `webView.canGoBack ? webView.goBack() : pop/dismiss`。

跨端框架（Flutter / RN 等）：URL 判断在原生 `WKNavigationDelegate` 里完成后，通过 Channel 把显隐 boolean 推给 UI 层即可，不需要在跨端层拦截 URL。

2.2 自定义 `navigationDelegate` 时的支付 scheme 协作

`WKNavigationDelegate` 是单委托协议——接入方若像 2.1 那样设置自己的 `navigationDelegate`（做标题栏判断等），会整体覆盖 `attachWebView` 内聚的 SDK 拦截（`weixin://alipays://alipay://` 拉起外部支付 App 失效）。此时在自己的 `decidePolicyFor` 开头调用 SDK 门面协作：

```
func webView(_ webView: WKWebView, decidePolicyFor navigationAction:
WKNavigationAction,
              decisionHandler: @escaping (WKNavigationActionPolicy) ->
Void) {
    // ① 先交给 SDK 处理支付 scheme（返回 true 表示已拉起外部 App，须
    Cancel）
    if let url = navigationAction.request.url,
        XMEcoSDK.shared.handleExternalAppScheme(url: url) {
        decisionHandler(.cancel)
        return
    }
    // ② 再做容器自己的标题栏判断等逻辑
    ...
    decisionHandler(.allow)
}
```

不设置自定义 `delegate` 的接入方无需关心本节（SDK 默认拦截已生效）。

3. SDK 接口

3.1 播放控制

```
let audio = XMEcoSDK.shared.audioServiceRef
```

// playContent 完整参数:

albumId/trackId/listenPackageId/breakSecond/enableLocalBreakpointResume
(后 3 参可选)

```
audio?.playContent(albumId: "13396678", trackId: "72982155",
```

```
listenPackageId: "0", breakSecond: 0.0,
```

```
enableLocalBreakpointResume: true)
```

```
audio?.pause(); audio?.resume(); audio?.stop(); audio?.toggle() // 基本  
控制
```

```
audio?.next(); audio?.prev() // 切换
```

曲目

```
audio?.seekTo(timeMs: 30000) // 跳转
```

(毫秒)

```
audio?.setPlaybackSpeed(rate: 1.5) // 倍速
```

3.2 外部播放委托 (渠道禁用 native 播放时)

渠道 `nativePlaybackAllowed=false` (如 noetixrobotics) 时, SDK 不执行本地播放。H5 页面里用户点播放会被 SDK 拦截, 默认只发错误事件 (接入方收不到播放意图)。

若接入方需要用自己的播放器播放, 注册 `ExternalPlaybackProvider`, SDK 会在拦截时把 H5 传入的播放参数回调给接入方:



复制代码

// 实现 `ExternalPlaybackProvider` 协议 (单方法
`onExternalPlaybackRequested`)

```
class MyPlaybackProvider: ExternalPlaybackProvider {  
    func onExternalPlaybackRequested(request: ExternalPlaybackRequest) {
```

```
        // request 字段:
```

```
        // albumId      String      专辑 ID (必填)
```

```
        // trackId      String?     曲目 ID (H5 只指定专辑时为 nil)
```

```
        // trackTitle   String?     曲目标题
```

```
        // duration     Int64?      曲目时长, 秒
```

```
        // breakSecond  Int32?      断点续播位置, 秒
```

```
        //
```

```
        // 在此用自己的播放器播放, 例如:
```

```
        myPlayer.play(  
            albumId: request.albumId,
```

```
            trackId: request.trackId,
```

```
            title: request.trackTitle ?? "",
```

```
            seekToMs: Int64(request.breakSecond ?? 0) * 1000
```

```

    }
}

// 注册（传 nil 取消注册）
XMEcoSDK.shared.setExternalPlaybackProvider(MyPlaybackProvider())

```

说明：

- 回调在 SDK 拦截 native 播放后立即触发，接入方可在其中启动自己的播放器。
- 封面（cover_url）不在回调参数内——拦截发生在 SDK 拉取专辑详情之前，此时无封面。如需封面，接入方可用 `albumId` 自行查询。
- 未注册时维持原行为（H5 播放被拦截，接入方收不到意图）。

高级播放设置

复制代码

```

// playerTimedStop 完整参数：
sleepStyle/second/episodes/playStopEpisodes/isOpen
// PlayerSleepStyle 枚举 case：
thIRTY_MINS/FIFTEEN_MINS/SIXTY_MINS/CUSTOM_SECS/CUSTOM_MINS 等
audio?.playerTimedStop(sleepStyle: .thIRTY_MINS, second: 0, episodes: 0,
                        playStopEpisodes: false, isOpen: true)
let timer = audio?.getPlayerTimedStop() // TimerStateInfo (5 字段)

audio?.setAlbumSort(albumId: "13396678", isAsc: true)
audio?.setSkipPlay(albumId: "13396678", skipBeginning: 30, skipEnding:
15)
audio?.setPlaybackSpeed(rate: 1.5, albumId: "13396678",
useCurrentAlbum: true, apply: true)
audio?.getPlaybackSpeed(albumId: "13396678") { info, error in
    if let error = error { /* 失败处理 */ }
    /* PlaybackSpeedInfo */
}

let trackInfo = audio?.hasPreAndNextTrack() // PreAndNextTrackInfo

```

事件监听

复制代码

```

class MyListener: AudioPlayerListener {
    override fun onPlaybackStarted(playable: PlayableTrack?) {}
    override fun onPlaybackPaused(playable: PlayableTrack?) {}
    override fun onPlaybackCompleted() {}
    override fun onPlaybackError(errorMessage: String, errorCode:
Int32, track playable: PlayableTrack?) {}
    override fun onPlaybackProgressUpdate(position: Int64, duration:
Int64, percent: Double) {} // 秒
    // 共 15 个回调
}
audio?.addPlayerListener(MyListener())

```

状态查询

▼ 复制代码

```

audio?.isPlaying() // 返回 KotlinBoolean (桥接类型, 用
.boolValue 解包为 Bool)
audio?.getCurrentPlayable() // PlayableTrack?
audio?.getPlaylist() // [PlayableTrack]
audio?.getSpeed() // Double

```

3.2 播控门控

▼ 复制代码

```

XMEcoSDK.shared.setNativePlaybackAllowed(false) // 禁用 (false 时暂停
+2600)

```

音频焦点

▼ 复制代码

```

XMEcoSDK.shared.requestAudioFocus() // 返回 KotlinBoolean
(用 .boolValue 解包为 Bool)
XMEcoSDK.shared.abandonAudioFocus()
XMEcoSDK.shared.setAudioFocusHeld(hold: true) // true=请求, false=释
放

```

3.3 登录

// 状态查询 (suspend→completion)

```
XMEcoSDK.shared.checkLoginStatus { status, _ in /* LoginStatus */ }
XMEcoSDK.shared.isUserLogin { isLogin, _ in }
XMEcoSDK.shared.isVipUser { isVip, _ in }
XMEcoSDK.shared.getUserNickname { name, _ in }
XMEcoSDK.shared.getUserAvatarUrl { url, _ in }
XMEcoSDK.shared.getUserId { uid, _ in }
```

// 登录 (两种方式, 按渠道接入约定选用)

// ① 票据登录: ticket 直接换取 token

```
XMEcoSDK.shared.loginWithThirdPartyTicket(ticket: "xxx") { result, _ in
}
```

// ② 三方 token 登录: SDK 内部将 ticket 拼成 {"ticket":"..."} 作为 third_info 透传

```
XMEcoSDK.shared.loginWithThirdPartyTokenByTicket(ticket: "xxx") {
result, _ in }
```

// 自动确保登录 (未登录时取 deviceInfoProvider 返回的三方信息 JSON 透传登录)

// deviceInfoProvider 返回 JSON 字符串, 字段同 getThirdDeviceInfo (device_id/ticket/model)

```
XMEcoSDK.shared.ensureLoginWithThirdPartyAuth(
    deviceInfoProvider: {
        let info: [String: Any] = ["device_id": ThirdParty.deviceId,
            "ticket": ThirdParty.ticket ?? ""]
        guard let data = try? JSONSerialization.data(withJSONObject:
info),
            let str = String(data: data, encoding: .utf8) else {
return "{}" }
        return str
    },
    callback: { result in /* LoginResult: success/loginType/message */ }
)
```

// ③ 登录并按权益状态决定是否建立登录态 (耦合登录与权益领取, suspend→completionHandler)

// 适用场景: 接入方希望在登录时校验是否有生态权益, 无权益则不建立登录态

```
XMEcoSDK.shared.loginAndClaimIfNeeded(
    loginInfoJson: ""{"ticket":"xxx"}"", // 登录凭据 JSON, 同
loginWithThirdPartyToken 的 third_info
```



```

// ticket（此处ticket是手机号加密后的登录票据）
vipDeviceInfo: nil
// 权益查询设备信息 JSON，为空时回退 VipTicketProvider.getVipDeviceInfo()
// 含 device_id + ticket
（此处ticket是设备SN加密后的票据）
){ result, _ in
    // result: LoginAndClaimResult
    // result.loginResult.success → 是否已建立/保持登录态
    // result.benefitQuery?.status → 权益状态（0..4），nil=未查到/查询失败
    // result.benefitQuery?.claimed → 是否触发领取（仅 status==4）
    // result.benefitQuery?.claimSuccess → 领取结果（claimed=true 时有意义）
}

// 监听器
XMEcoSDK.shared.addLoginStatusListener(listener)

// 登录成功数据透传（实际登录成功时触发，影子登录除外）
// 覆盖：SDK 主动登录（loginWithThirdPartyTicket/loginAndClaimIfNeeded 转正）+ H5 同步登录（updateAuthEntity）
// 影子登录（shadowLogin，仅查权益的临时 token，不落库）不触发
XMEcoSDK.shared.setLoginDataListener { json in
    // json:
    {"access_token":"...", "refresh_token":"...", "uid":3192, "login_type":"THIRD_PARTY"}
    // login_type: TICKET / MOBILE / THIRD_PARTY（区分登录来源，便于埋点）
}

```

3.4 权益注入

注册 VIP 票据提供者，SDK 在权益查询/领取时委托回调。

`VipTicketProvider` 是 ObjC 协议，方法为异步回调式——每个分支都必须调用 **completionHandler**，否则回调永不返回导致 H5 桥超时。

各方法说明：

方法	必填	completionHandler 入参	说明
----	----	-------------------------	----

getVipTicket	✓	String? (JWT 票据)	H5 通过 JS Bridge 取 VIP 票据，未注入时报错
getVipDeviceInfo	○	String? (设备信息 JSON 字符串)	本机设备信息，默认 "{}"，回传 nil 由 SDK 兜底为 "{}"
getThirdDeviceInfo	○	String? (三方设备信息 JSON 字符串)	三方设备信息，默认 "{}"，回传 nil 由 SDK 兜底为 "{}"
onVipClaimResult	✓	— (无 completionHandler)	权益领取结果回调，result 为结果 JSON 字符串。fetchEcoBenefitQualification(autoClaim:true) 自动领取的结果也走此回调 ({"success":bool,"code":int,"msg":"..."})

⚠ getVipDeviceInfo / getThirdDeviceInfo 的 completionHandler 回传的是 **JSON 字符串** (如 "{"device_id":"xxx"}")，不是 Dictionary。用 JSONSerialization 将字典转字符串后回传。

JSON 字段格式 (接入方按渠道需要填充，无需的字段可不填)：

方法	字段	类型	示例	说明
getVipDeviceInfo	device_id	String	"DILZUDID5504A DYC257M0010"	本机设备唯一标识
	ticket	String	"eyJ0eXA..."	VIP 票据 (与 getVipTicket 同源)


```

        completionHandler(nil, nil)
        return
    }
    let info: [String: Any] = [
        "device_id": ThirdParty.deviceId,
        "ticket": ticket,
        "model": Self.deviceModel,
    ]
    completionHandler(Self.toJsonString(info), nil)
}

// 【必填】权益领取结果回调。result 格式: {"success":true,"msg":"领取成功"}
func onVipClaimResult(result: String) {
    guard let data = result.data(using: .utf8),
        let json = try? JSONSerialization.jsonObject(with: data)
    as? [String: Any] else {
        NSLog("[Vip] 解析领取结果失败: \(result)")
        return
    }
    NSLog("[Vip] 领取结果: success=\(json["success"] ?? false), msg=\(json["msg"] ?? "")")
}

// 字典 → JSON 字符串（序列化失败时兜底 "{}"）
private static func toJsonString(_ dict: [String: Any]) -> String {
    guard let data = try? JSONSerialization.data(withJSONObject: dict),
        let str = String(data: data, encoding: .utf8) else {
    return "{}" }
    return str
}

private static let deviceModel: String = {
    var systemInfo = utsname()
    uname(&systemInfo)
    return withUnsafePointer(to: &systemInfo.machine) { ptr in
        String(cString:
UnsafeRawPointer(ptr).assumingMemoryBound(to: CChar.self))
    }
}()

```

```
private func isExpired(_ jwt: String) -> Bool { /* 解析 JWT exp 判断 */ false }
}
```

// 注册（必须在 `initialize` 之后、进入权益 H5 页之前；通常放 `AppDelegate.didFinishLaunching`）
`XMEcoSDK.shared.setVipTicketProvider(provider: MyVipProvider())`

💡 **时序：**必须 `XMEcoSDK.shared.initialize()` 之后再注入；建议在 `AppDelegate.didFinishLaunching` 或进入权益页的 `viewDidLoad` 注入，不要等用户点击才注入。

3.5 数据获取

通过 `sdk?.businessService`（suspend→completion）：

📄 复制代码

```
let biz = XMEcoSDK.shared.sdk?.businessService
biz?.fetchCommonSubscribeList(page: 1, limit: 10) { list, _ in /* [AlbumInfo] */ }
biz?.subscribeAlbum(albumId: "13396678") { ok, _ in }
biz?.unsubscribeAlbum(albumId: "13396678") { ok, _ in }
biz?.isSubscribeBatch(albumIds: "1,2,3") { map, _ in /* [String: Bool] */ }
biz?.fetchCommonHistoryList(page: 1, limit: 10) { list, _ in }
biz?.fetchListenList(page: 1, limit: 10, ids: "1,2") { map, _ in }
biz?.fetchAlbumBatch(ids: "1,2,3") { list, _ in }
biz?.textAiSearch(text: "三国", limit: "10") { resp, _ in } // limit 为 String
biz?.fetchEcoBenefitQualification(thirdDeviceInfo: nil, autoClaim: false) { data, _ in } // autoClaim:true 时查询到可领权益自动领取，结果回调 onVipClaimResult
biz?.fetchDeviceVipInfo() { info, _ in } // info.subscribe_activity 为权益领取页 URL
// 权益判断: data?.ecological_benefit_status == 4 (EcoBenefitStatus.HAS_BENEFIT) 表示有权益可领取
biz?.verifyAlbumPermission(albumId: "13396678") { ok, _ in }
```

搜索接口（`searchAlbum`）属于 `ContentService`：`XMEcoSDK.shared.sdk?.contentService?.searchAlbum(keyword: "三国", page: 1, pageSize: 20) { page, _ in }`，返回 `ContentPage<TrackItem>`。

`fetchEcoBenefitQualification` 参数说明：

参数	类型	默认值	说明
<code>thirdDeviceInfo</code>	<code>String?</code>	<code>nil</code>	三方设备信息 JSON 字符串 （如 <code>{"device_id":"xx"}</code> ）。为空时 SDK 自动从 <code>VipTicketProvider</code> 获取
<code>autoClaim</code>	<code>Boolean</code>	<code>false</code>	<code>true</code> 时查询到可领权益（ <code>status==4</code> ）自动领取，结果回调 <code>onVipClaimResult</code> ； <code>false</code> 时仅查询

3.5.1 登录并领取权益（loginAndClaimIfNeeded）

`loginAndClaimIfNeeded` 把"登录"和"权益领取"耦合为原子操作，按当前登录态分两条路径：

未登录路径（核心场景）：

- 影子登录：拿到 token + uid，不落库、不通知监听器、不切状态机（临时态，仅用于查权益）
- 用影子 token 查询权益资格：
 - 查询技术失败 → 不登录，返回失败
 - `status==4`（有权益）→ 正式登录 + 自动领取
 - `status==0`（无权益）→ 不建立登录态，返回失败
 - 其他状态（1/2/3）→ 正式登录

已登录路径（用户已登录后再调用）：

- 跳过影子登录，用现有 token 查询权益（不覆盖当前登录态），每次调用都回调 `setLoginDataListener`
- `status==4` → 自动领取，保持登录态
- 其他状态 → 直接放行（不登登）

⚠ 并发安全：`loginAndClaimIfNeeded` 内部用 Mutex 串行化，并发调用会排队执行。SDK 为单全局登录态，多设备连续调用时第二次走已登录路径用的是第一次设备的 token。多

设备切换建议先 `logout()` 再调本方法。

💡 **领取结果双通道**：领取结果既回调 `VipTicketProvider.onVipClaimResult`（与 `fetchEcoBenefitQualification` 一致），也通过返回值 `benefitQuery.claimSuccess` 提供。

返回字段	类型	说明
<code>loginResult.success</code>	<code>Boolean</code>	是否已建立/保持登录态
<code>benefitQuery</code>	<code>EcoBenefitQueryResult?</code>	<code>nil</code> =未进入查询/查询技术失败
<code>benefitQuery.status</code>	<code>Int</code>	权益状态 0..4
<code>benefitQuery.claimed</code>	<code>Boolean</code>	是否触发领取（仅 <code>status==4</code> ）
<code>benefitQuery.claimSuccess</code>	<code>Boolean?</code>	领取结果；未领取时为 <code>nil</code>

参数说明：

参数	类型	说明
<code>loginInfoJson</code>	<code>String</code>	登录凭据 JSON，透传为登录接口的 <code>third_info</code> 。含 <code>ticket</code> （手机号加密后的登录票据），如 <code>"{\\"ticket\\":\\"eyJ0eXA...\\"}"</code>
<code>vipDeviceInfo</code>	<code>String?</code>	权益查询设备信息 JSON，为空时回退 <code>VipTicketProvider.getVipDeviceInfo()</code> 。含 <code>device_id</code> + <code>ticket</code> （设备 SN 加密后的 VIP 票据），如 <code>"{\\"device_id\\":\\"xxx\\",\\"ticket\\":\\"eyJ0eXA...\\"}"</code>

⚠️ **两个 ticket 含义不同**：`loginInfoJson` 的 `ticket` 是手机号加密的登录票据（用户维度）；`vipDeviceInfo` 的 `ticket` 是设备 SN 加密的 VIP 票据（设备维度，与 `getVipTicket()` 同源）。

3.6 悬浮播放器

[复制代码](#)

```
FloatingPlayerManager.shared.show()
FloatingPlayerManager.shared.hide()
FloatingPlayerManager.shared.isShowing()
FloatingPlayerManager.shared.toggle() // 播放/暂停
```

3.7 页面跳转 / 埋点

每条音频播放结束（切换/暂停/完成/列表完成）时回调本次累计播放时长：

[复制代码](#)

```
XMEcoSDK.shared.addCommonActionListener { action, albumId, trackId in
/* "openPlayer"/"openAlbumDetail" */ }
XMEcoSDK.shared.addAnalyticsListener { event in /*
albumId/trackId/playSeconds（本次净播放秒数，已扣除暂停）/startAt */ }
```

3.8 支付

[复制代码](#)

```
XMEcoSDK.shared.setPaymentType(1) // 0=SDK支付, 1=接入方支付
XMEcoSDK.shared.getPaymentType() // Int
XMEcoSDK.shared.addHostPaymentListener { orderId, amount, productId,
params in /* 拉起支付 */ }
XMEcoSDK.shared.addPaymentListener { result in /* HostPaymentResult:
code/message */ }
// 回灌支付结果（code: 0=成功）
XMEcoSDK.shared.notifyHostPaymentResult(HostPaymentResult(code: 0,
message: "成功"))
```

3.9 渠道信息

[复制代码](#)

```
XMEcoSDK.shared.getChannelName(); XMEcoSDK.shared.getAppKey();
XMEcoSDK.shared.getSN()
XMEcoSDK.shared.getChannelId()
```

```
XMEcoSDK.shared.isPlayingEnabled();
XMEcoSDK.shared.isEcoLinkModuleAvailable()
XMEcoSDK.shared.isAIModuleAvailable();
```

3.10 URL 构建

[复制代码](#)

```
XMEcoSDK.shared.buildDefaultPlayerUrl(albumId: "13396678", trackId:
"72982155", breakSecond: 0) // H5 播放页
XMEcoSDK.shared.buildAlbumDetailUrl(albumId: "13396678") // H5 专辑详情
页
```

4. ObjC 命名映射

SDK 类在 ObjC 层带 `XMESDK` 前缀，Swift 中直接用去掉前缀的名字：`XMEcoSDK.shared`、`AudioService`、`PlayableTrack` 等。

异步方法自动带 completion handler。返回 Bool 时需用 `.boolValue` 解包。

5. 错误码

码	说明	处理建议
1000	未知错误	检查参数和网络
1001	网络错误	检查网络连接
2000	播放错误	检查音频资源有效性
2019	音频列表为空	该专辑暂无可播放内容
2026	播放鉴权错误	检查用户权限/登录状态
2501	非版权内容	该内容暂时无法播放
2600	原生播控被禁用	<code>setNativePlaybackAllowed(true)</code> 后重试
3000	登录失败	检查登录凭据

3001	令牌已过期	重新登录
3006	用户未登录	引导用户登录

6. 注意事项

1. 初始化：必须在 `AppDelegate.didFinishLaunching` 调用，幂等。
2. 版本号：SDK 版本 `2.0.0` 不可修改。
3. 权益：`setVipTicketProvider` 的 `getVipTicket / onVipClaimResult` 必填；`getVipDeviceInfo / getThirdDeviceInfo` 可选（回传 JSON 字符串，默认 `"{}"`，回传 nil 由 SDK 兜底为 `"{}"`）。
4. SDK 生命周期与登录态管理（重要）：
 - 退出登录只需调 `XMEcoSDK.shared.logout { _ in }`（或 H5 调 `AuthBridge.logout`），清除 token + 通知监听器。
 - 关闭 H5 页面只需 `detachWebView()` 断开桥 + 销毁 WebView。
 - `XMEcoSDK.shared.release()` 是 SDK 彻底销毁（销毁播放器、关闭网络引擎、不可逆），仅在 App 完全退出 SDK 功能模块时调用。退出登录、切换账号、关闭 H5 页面都不要调 `release()`。
 - `VipTicketProvider` 的 `suspend` 方法映射为 `completionHandler`——每个分支都必须调用 `completionHandler`，否则协程永久挂起导致 H5 桥超时。如果 `completionHandler` 内有异步网络请求，务必设超时兜底。